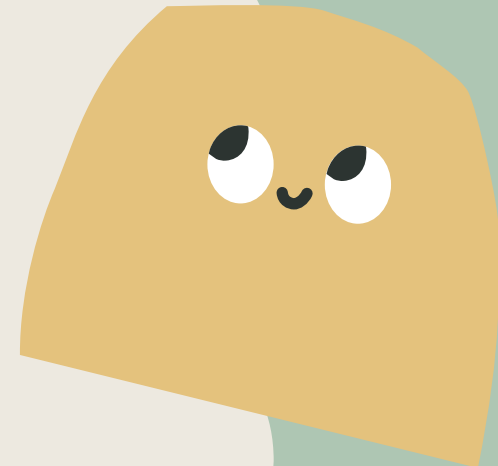
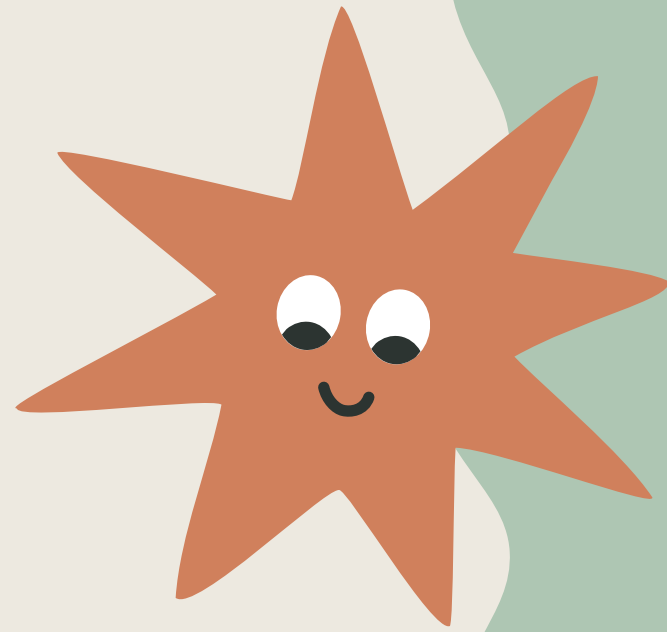


Complexity Analysis, Time Complexity & Space Complexity

Lecture 3



Learning Outcomes (Lecture 3)

➤ At the end of this lecture, students will be able to:

- Explain why complexity analysis is necessary.
- Compare two algorithms based on efficiency.
- Differentiate between execution time and time complexity.
- Explain time complexity.
- Explain space complexity.
- Identify factors affecting program performance.
- Understand the idea behind asymptotic analysis.
- Recognize common growth rates of algorithms.

Real World Examples

Example 1

- Suppose Google searches 100 Billion web pages
- Would Google compare every webpage one by one? (Impossible)
- Efficiency matters more than correctness when dealing with huge data

Example 2

- Imagine two routes from Gujrat to Lahore
- Route A distance 150 KM and Route B distance 170 KM
- Which is better? (Depends)
- Maybe Route B is faster because of the motorway
- Similarly Shortest code $\neq$ Fastest code

What is Complexity Analysis?

- Complexity Analysis is the process of measuring the efficiency of an algorithm in terms of the resources it requires
- Resources include
 - Time
 - Memory
- Why analyze algorithms?
 - Suppose algorithm A takes 1 second and algorithm B takes 10 seconds
 - Which one should Google use? **Obviously, algorithm A**
- But Wait...
 - Suppose algorithm A runs on core i9 algorithm B runs on core i3
 - Can we compare them? **NO**

Two Types of Analysis

Machine-Dependent Analysis

- Depends on
 - CPU, RAM, Compiler, Operating System
 - Bad method

Machine-Independent Analysis

- Instead of measuring seconds we count
 - Number of Operations
- This is called **Complexity Analysis**

Example

- Algorithm A
 - 10 operations
- Algorithm B
 - 100 operations
- Regardless of computer
- Algorithm A is better

Time Complexity

- Time Complexity measures how the number of operations increases as the input size increases

Important

- It does **NOT** measure seconds
- It measures **Growth**

Type 1: Constant Growth $O(1)$

Real life example

- Imagine a class with 10 students and teacher asks
- What is today's date?
- We get answer in one operation
- Now consider a class with 100 students and teacher asks same question
- How many operations it will take to get answer?
- This is called **Constant Growth**
- Programming example:
- Accessing one array element

```
cout << arr[5];
```

Type 2: Linear Growth $O(n)$

Real life example

- Suppose you're asked to count students in a classroom
- For 10 students you count from 1 to 10. Total operations = 10
- For 100 students you count from 1 to 100. Total operations = 100
- When input (students) becomes 10 times larger
- Work also becomes 10 times larger
- This is called **Linear Growth**
- Programming example:
- Single loop

```
for(int i=0;i<n;i++)
```

Type 3: Quadratic Growth $O(n^2)$

Real life example

- Suppose every student shakes hands with every other student
- With 10 students, total operations $10 \times 10 = 100$
- With 100 students, total operations $100 \times 100 = 10000$
- Students increase 10 times, but operations increase 100 times
- This is called **Quadratic Growth**
- Programming example:
- Nested loop

```
for(i=0;i<n;i++)  
{  
    for(j=0;j<n;j++)
```

Type 4: Logarithmic Growth $O(\log n)$

Real life example

- I choose a random number between 1 and 1000
- You've to guess the number
- If I tell you higher, lower
- You keep removing half the possibilities
- Very few questions needs
- This is called **Logarithmic Growth**
- Programming example:
- Binary Search

Type 5: Exponential Growth $O(2^n)$

Real life example

- Suppose every student invites 2 friends
- Each friend invites 2 more
- Very quickly population explodes
- $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \dots$
- This is called **Exponential Growth**
- Programming example:
- Fibonacci series

```
int fibonacci(int n)
{
    if (n <= 1)
        return n;

    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Compare Growths

Suppose input = 1000

Complexity	Approximate Operations
$O(1)$	1
$O(\log n)$	10
$O(n)$	1000
$O(n \log n)$	10000
$O(n^2)$	1000000
$O(2^n)$	unimaginably huge

Space Complexity

- Space Complexity measures how much memory an algorithm requires as the input size grows
- Notice the similarity
- Time Complexity → How work grows
- Space Complexity → How memory grows

Type 1: Constant Space $O(1)$

Real life example

- Suppose a teacher always carries:
 - One marker
 - One duster
- Even if the class has 10 students or 100 students
- The teacher still carries only these three items
- This is called **Constant Growth**
- Programming example:
- Always three variables

```
int a;  
int b;  
int sum;
```

Type 2: Linear Space $O(n)$

Real life example

- Suppose every student submits one assignment
- We need one file for one student
- 10 students 10 files, 100 students 100 files and so on
- Memory grows with students
- This is called **Linear Growth**
- Programming example:
- Need one memory location for every element

```
int marks[n];
```

Type 3: Quadratic Space $O(n^2)$

Real life example

- Suppose every student rates every other student
- For 10 students we've to create a table of $10 \times 10 = 100$ cells
- With 100 students, total cells $100 \times 100 = 10000$ cells
- Memory increased much faster
- This is called **Quadratic Growth**
- Programming example:
- Matrix declaration

```
int matrix[n][n];
```

Time Complexity vs Space Complexity

- Many students think
- If an algorithm has $O(n)$ Time Complexity, then its Space Complexity must also be $O(n)$
- This is wrong
- For example:

```
int sum = 0;

for (int i = 0; i < n; i++)
{
    sum += arr[i];
}
```

- **Time Complexity:** $O(n)$ (the loop runs n times)
- **Space Complexity:** $O(1)$ (only `sum` and `i` are extra variables)